

BE ↔ FE 接口规范

位宽不在本文档定义,由实现方决定。

1. 接口一览

用途	方向	信号
指令交付	FE → BE	fe_be_instr_valid / fe_be_instr_pld[0:1]
实收回执	BE → FE	be_fe_instr_accept
恢复取指	BE → FE	be_fe_redirect_valid / _pc / _kind / be_fe_icache_invalidate
预测器训练	BE → FE	be_fe_bp_update_valid / _pc / _taken / _target / _cf_class

global_flush_late 不连 FE。FE 只消费 be_fe_redirect_*。

命名约定

fe_be_*	由 FE 驱动、BE 采样	前缀 = 该线自身方向，必须与 RTL 端口 input/output 一致
be_fe_*	由 BE 驱动、FE 采样	

端口名 ≠ 类型名: fe_be_instr_pld 是端口,类型是 IB_Payload。

be_fe_instr_accept 不是 ready,是 valid ∧ 有位 ∧ !flush 的实收结果。

2. 端口表

方向	信号	含义
FE → BE	fe_be_instr_valid	本拍候选 slot
FE → BE	fe_be_instr_pld[0] / [1]	slot0 / slot1 的已译码指令
BE → FE	be_fe_instr_accept	本拍实收 slot
BE → FE	be_fe_redirect_valid	本拍发生恢复取指
BE → FE	be_fe_redirect_pc	恢复目标;仅 valid=1 时有效
BE → FE	be_fe_redirect_kind	恢复原因
BE → FE	be_fe_icache_invalidate	取指侧缓存失效请求
BE → FE	be_fe_bp_update_valid	一条控制流指令已解析
BE → FE	be_fe_bp_update_pc	该指令 PC
BE → FE	be_fe_bp_update_taken	实际方向
BE → FE	be_fe_bp_update_target	实际 taken 目标
BE → FE	be_fe_bp_update_cf_class	控制流类别

位编码

fe_be_instr_pld[0] = 程序序第一条, [1] = 其紧邻下一条。两个向量同一套位序:

取值	fe_be_instr_valid	be_fe_instr_accept
00	无候选	未收
01	仅 slot0	仅收 slot0
11	slot0 + slot1	两条都收
10	非法	非法

只能表达连续前缀,不存在"只给 slot1"或"只收 slot1"。顶层直连,无位序映射。

3. 时序口径

项目	强制口径
be_fe_instr_accept	与本拍 fe_be_instr_valid / payload 对应的组合实收结果
接受判据	含 IB 本拍 dequeue 释放的空位
容量提示	后端 不导出 ; be_fe_instr_accept 是唯一回压通路
redirect	优先于交付,当拍 be_fe_instr_accept 强制 00
采样沿	不限定

4. 指令交付协议

FE 盲发 → 读回实收 → 滑窗:

accept = 11	两条均收;指针 +2, 下拍给新候选
accept = 01	仅 slot0 收;原 slot1 移为新 slot0, 尾部补一条
accept = 00	一条未收;候选组保持, 下拍原样重发

5. Redirect

be_fe_redirect_kind	名称	be_fe_redirect_pc
0	MISPREDICT	分支的恢复 PC
1	EXCEPTION	trap 向量
2	MRET	mepc
3	INTERRUPT	trap 向量
4	FENCE_I	已提交 FENCE. I 的下一条 PC
5-7	保留	不得使用

be_fe_redirect_valid = 1 时 FE 的动作:

1	不执行滑窗 / 压缩 / 补充
2	作废全部候选, 含前拍被拒待重发的;保持义务同时解除
3	锁存 be_fe_redirect_pc, 下拍从该地址重取
4	连续 redirect 时最近一次覆盖旧目标

be_fe_icache_invalidate 是**独立信号**,不是 be_fe_redirect_kind 的本地译码结果。为 1 时,FE 必须在从 be_fe_redirect_pc 重取之前完成取指侧缓存状态的失效。当前等价于 kind == FENCE_I ,两根线仍须各自完整连接。

6. 预测器训练

`be_fe_bp_update_*` 由 G0 · BRU 在**执行解析拍**广播,不等提交。与 `redirect` 不互斥:预测正确时单独出现,预测错误时可与 `redirect` 同拍;被后续 flush 的控制流也可能已训练。**无提交侧重训通路。**

<code>cf_class = 00</code>	<code>COND_BRANCH</code>	BEQ/BNE/BLT/BGE/BLTU/BGEU、C.BEQZ、C.BNEZ
<code>01</code>	<code>DIRECT_JUMP</code>	JAL、C.J
<code>10</code>	<code>INDIRECT_JUMP</code>	JALR、C.JR、C.JALR
<code>11</code>	保留	

`DIRECT_JUMP` / `INDIRECT_JUMP` 的 `actual_taken` 恒为 1,不构成方向样本。

`be_fe_bp_update_target` 与 `be_fe_redirect_pc` 语义不同,不可混用:

<code>be_fe_bp_update_target</code>	该指令实际 taken 时的控制流目标;taken = 0 时无意义,不得采样
<code>be_fe_redirect_pc</code>	<code>actual_taken ? target : branch_pc + 指令长度</code>

指令长度由 FE 自己的 `is_compressed` 得知,后端不回送。

本接口不提供 RAS 信息。 BRU 的执行 payload 没有 `rd` / `rs1` 寄存器号,无法识别 call/return。将来实现 RAS 须在派遣侧传入 `ras_action` 或补寄存器号。

7. Reset 与停止

<code>reset active</code>	FE 清零 <code>fe_be_instr_valid</code> 与全部 payload BE 回送 <code>accept = 00</code> 、 <code>redirect_valid = 0</code>
指令流结束	FE 不再产生新 valid;已给出候选仍按 <code>accept</code> 完成
外部停止	FE 在下一驱动沿清零输出

复位 PC 是系统 / 顶层边界,不由本接口定义。

8. IB_Payload 字段表

字段	语义
pc	本条指令地址;trap epc、AUIPC 基址、BRU fall-through 均以此为准
inst_bits	原始编码;RVC 只用低 16 位、高位清零。非法指令 tval 的唯一来源
is_compressed	是否 16-bit 指令
is_serial	见 §8.1
is_fp_instruction	触碰 FP 寄存器堆,含 FP load/store
rs1_idx / rs2_idx / rs3_idx	源寄存器号
use_rs1 / use_rs2 / use_rs3	对应源是否参与
rs1_is_fp / rs2_is_fp / rs3_is_fp	对应源属 FP 还是整数堆
rd_idx	目的寄存器号
use_rd	是否定义目的结果; x0 写抑制由后端处理,FE 照实填
rd_is_fp	目的属 FP 还是整数堆
is_store	仅普通缓冲 store 的 drain 标志;原子指令恒 0
mem_funct3	访存宽度 / 整数符号扩展 / FP 类型的共同编码
imm_valid / imm_data	已译码立即数(非原始编码);CSR 立即数型的 uimm 也走这里
pred_taken / pred_target_pc	FE 的预测结果;仅 BRU 消费
exe_subop	已译码指令 ID;后端分组、FU 选择、FU 操作译码的唯一来源
full_decode	CSR / 非法 / FP 舍入的补充控制字段

8.1 is_serial

= 1	CSR 指令、MRET、FENCE、FENCE.I、全部 22 条原子指令(LR/SC/9 种 AMO 各 .W/.D)
= 0	ECALL、EBREAK、WFI —— 架构效应在按序退休处处理

is_store = 1 仅普通 / FP store。原子指令、FENCE、FENCE.I 恒 0。

FP load/store 必须同时置 is_fp_instruction ,并按源 / 目的设置 FP 位。

8.2 exe_subop / full_decode 位布局

```
exe_subop[23:22] = format          01 普通 32-bit;10 RVC
exe_subop[21:15] = opcode_or_op
exe_subop[14:12] = funct3
exe_subop[11:0]  = high_fixed      FP .S/.D 精度由此段区分

full_decode[16]   = csr_write_intent
full_decode[15]   = illegal
full_decode[14:12] = rm[2:0]       000..100 = RNE/RTZ/RDN/RUP/RMM;111 = DYN;101/110 保留
full_decode[11:0] = csr_addr[11:0]
```

子码全集以 subop/exe_subop_pkg.sv 为准。不适用字段置零并忽略。

8.3 csr_write_intent

由 decode 产生,不得由后端从 rs1_data 推导:

```
1  CSRRW / CSRRWI
1  CSRRS / CSRRC  且 rs1_idx != x0
1  CSRRSI / CSRRCI 且 uimm != 0
0  其余(含非 CSR 指令)
```

判据是寄存器号 rs1_idx == x0,不是读出值 rs1_data == 0。

8.4 illegal

illegal = 1 时 exe_subop 必须置 SUBOP_INVALID。适用范围:

```
编码本身非法
ENABLE_A = 0 时的原子子码
ENABLE_C = 0 时的 RVC 子码
```

这类指令必须照常交付、走 ILLEGAL 完成路径报 cause 2,不得在 FE 侧拒绝交付。

FP 的 FS == Off、保留 rm、DYN 对应保留 frm 由后端派遣侧判定;decode 照实提供 is_fp_instruction 与 rm。